



Checkpoint 1

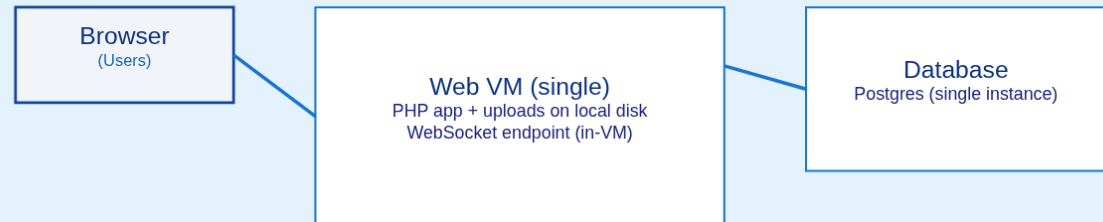
Project 0 Analysis

Cloud Computing and Virtualization

Beatriz Moreira N°25262

Project O (Original)

- Simple PHP web application (Project O)
- Single web VM serving PHP/HTML/JS
- Local filesystem for uploads
- Single relational DB (Postgres)
- WebSocket endpoint on the same single web VM (current/monolithic architecture)



Legend

- Monolithic: single web VM with integrated WebSocket and local storage
- Database: single instance (no replication)

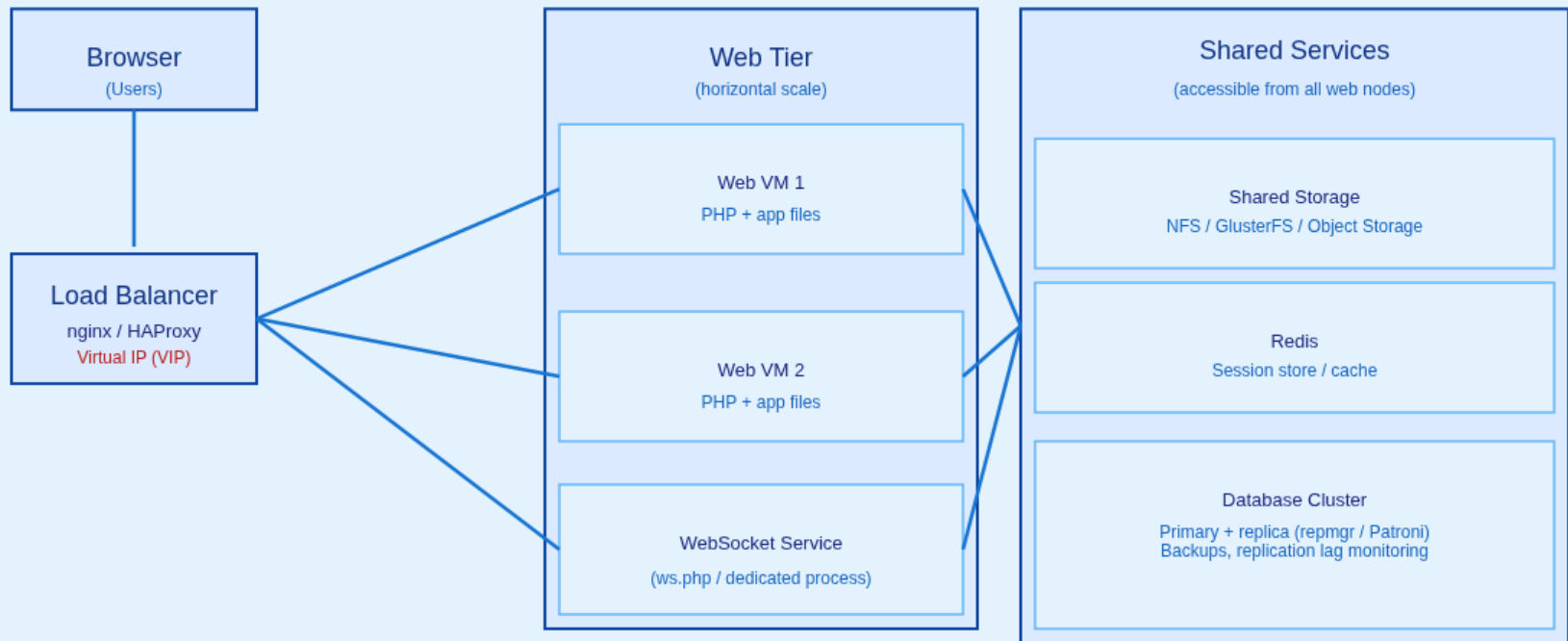
Main Limitations

- Single web server: no horizontal scaling or redundancy
- Local sessions: not shareable across nodes
- Local uploads: not durable or accessible by multiple nodes
- Single DB: single point of failure; no HA
- WebSocket on web VM: hard to scale
- No load balancer, no automated provisioning, limited monitoring

Initial Requirements

- Horizontal web tier (multiple web VMs)
- Centralized sessions (Redis)
- Shared storage for uploads (NFS/Gluster or object storage)
- DB replication and failover (repmgr or Patroni)
- Highly-available load balancer (nginx/HAProxy + Keepalived)
- WebSocket handling embedded in each Web VM (workers) + Redis Pub/Sub for cross-instance messaging; LB sticky sessions when needed
- Automation with Ansible + Vagrant
- Basic monitoring (Prometheus + Grafana)

Simple Architecture Diagram



Legend

- Load Balancer: endpoint; HA via Keepalived
- Web Tier: horizontally scalable PHP app servers
- Shared Services: Redis, shared storage, and DB cluster for HA
- Sessions: centralized (Redis)

Initial Plan for Solution A (Vagrant/VMs)

1. Define VM roles in `Vagrantfile`: LBs, web nodes, storage, Redis, DB cluster, WS nodes.
2. Provision with Ansible: install PHP, web server, mount shared storage, configure Redis and DB replication.
3. Configure Keepalived on LBs for virtual IP and health checks.
4. Make PHP app stateless: switch session handler to Redis; update `public\_html/\_dotenv.php` if needed.
5. Move uploads to shared storage and update `upload.php`.
6. Configure DB replication + failover.
7. Route WebSocket traffic: keep WebSocket workers embedded in each Web VM, use LB sticky sessions and Redis Pub/Sub to propagate messages between instances.
8. Add monitoring (node exporter, Prometheus, Grafana).
9. Test scaling and failover (simulate node failures).